Tribe RISC-V CPU

About

Tribe is a RV32 RISC-V CPU model written in the CppHDL C++ dialect. The model is intended to run both as a native C++ simulation and as generated SystemVerilog through Verilator. The core implements a small in-order pipeline with instruction and data L1 caches, a shared L2 cache, CSR/trap support, optional interrupt support, optional Sv32 MMU/TLB support, coherent AXI4-style memory ports, and an SoC wrapper with memory-mapped devices.

The base implementation targets 32-bit integer software. Build-time configuration in `tribe/Config.h` selects optional blocks such as `ENABLE_ZICSR`, `ENABLE_RV32IA`, `ENABLE_ISR`, and `ENABLE_MMU_TLB`. The L2 memory data width is selected with `L2_AXI_WIDTH`; common test targets use 64, 128, and 256 bits. Main memory starts at `memory_base_in`, has runtime size `memory_size_in`, and is split into four L2 memory/device regions. The last region is used as uncached IO/MMIO space.

Structure

The top-level CPU is the `Tribe` module in `tribe/main.cpp`. It instantiates these core blocks:

Block	Source	Role
`Decode`	`tribe/Decode.h`	Instruction decode, register source selection, and `State` construction.
`Execute`	`tribe/Execute.h`	ALU operation and branch resolution.
`ExecuteMem`	`tribe/ExecuteMem.h`	Memory request generation, split access handling, and optional atomics.
`WritebackMem`	`tribe/WritebackMem.h`	Load response capture, split-load assembly, and store-to-load forwarding.
`Writeback`	`tribe/Writeback.h`	Architectural register writeback formatting.
`CSR`	`tribe/CSR.h`	CSR, privilege, trap, and return-from-trap state.
`MMU_TLB`	`tribe/MMU_TLB.h`	Optional Sv32 instruction/data address translation and page-table walking.

Block	Source	Role
`InterruptController`	`tribe/InterruptController.h`	Optional CLINT and PLIC/external interrupt routing into CSR trap input.
`File<32,32>`	`File.h`	Integer register file.
`L1Cache`	`tribe/cache/L1Cache.h`	Separate instruction and data L1 caches.
`L2Cache`	`tribe/cache/L2Cache.h`	Shared coherent L2 cache, AXI memory/device master ports, and external AXI slave ports.
`BranchPredictor`	`tribe/BranchPredictor.h`	Small direct-mapped branch predictor.

The executable test wrapper in `tribe/main.cpp` instantiates three RAM regions, an IO region mux, NS16550A UART, CLINT, PLIC, and Accelerator. `tribe/SoC/System.cpp` packages the same CPU and devices into a synthesizable-style `System` module where the first two DRAM regions remain outside the DUT and the third RAM/device region is inside the SoC.

Main (Core)

`Tribe` is a four-stage (including i-fetch) in-order CPU core plus a fetch path. The pipeline state array tracks Decode-to-Execute and Execute-to-Writeback state; instruction fetch is driven directly from `pc`, I-cache output, branch prediction, and MMU translation. The `State` structure carries decoded instruction control fields through the core.

Top-level `Tribe` ports:

Port	Type	Description
`dmem_write_out`	`bool`	Debug/trace indication of a data-memory write.
`dmem_write_data_out`	`uint32_t`	Debug/trace write data for the current data-memory write.
`dmem_write_mask_out`	`uint8_t`	Debug/trace byte mask for the current data-memory write.
`dmem_read_out`	`bool`	Debug/trace indication of a data-memory read.
`dmem_addr_out`	`uint32_t`	Debug/trace current data-memory address.

Port	Type	Description
imem_read_addr_out	uint32_t	Debug/trace current instruction-memory address.
debug_immu_ptw_read_out	bool	MMU debug: IMMU page-table-walk read request, when MMU is enabled.
debug_immu_ptw_addr_out	uint32_t	MMU debug: IMMU page-table-walk address.
debug_immu_busy_out	bool	MMU debug: IMMU is walking or waiting.
debug_immu_fault_out	bool	MMU debug: IMMU fault state.
debug_immu_paddr_out	uint32_t	MMU debug: current IMMU translated physical address.
debug_icache_read_valid_out	bool	I-cache debug: read response valid.
debug_icache_read_addr_out	uint32_t	I-cache debug: response address tag.
debug_fetch_valid_out	bool	Fetch path has a valid instruction for decode.
debug_memory_wait_out	bool	Core memory path is stalling the pipeline.
debug_wb_load_ready_out	bool	Writeback memory stage has a completed load.
debug_wb_mem_wait_out	bool	Writeback-stage memory operation is still waiting.
debug_icache_read_in_out	bool	I-cache read request debug mirror.
debug_icache_stall_in_out	bool	I-cache stall input debug mirror.
debug_immu_last_addr_out	uint32_t	MMU debug: last IMMU PTE address.
debug_immu_last_pte_out	uint32_t	MMU debug: last IMMU PTE value.
debug_dmmu_ptw_read_out	bool	MMU debug: DMMU page-table-walk read request.
debug_dmmu_ptw_addr_out	uint32_t	MMU debug: DMMU page-table-walk address.
debug_dmmu_busy_out	bool	MMU debug: DMMU is walking or waiting.

Port	Type	Description
debug_dmmu_fault_out	bool	MMU debug: DMMU fault state.
debug_mmu_ptw_word_out	uint32_t	MMU debug: 32-bit PTE word selected from L2 data.
debug_pc_out	uint32_t	MMU debug: current architectural PC.
debug_satp_out	uint32_t	CSR debug: current satp.
debug_mstatus_out	uint32_t	CSR debug: current mstatus.
debug_mtvec_out	uint32_t	CSR debug: current mtvec.
debug_mepc_out	uint32_t	CSR debug: current mepc.
debug_mcause_out	uint32_t	CSR debug: current mcause.
debug_mtval_out	uint32_t	CSR debug: current mtval.
debug_sepc_out	uint32_t	CSR debug: current sepc.
debug_stvec_out	uint32_t	CSR debug: current stvec.
debug_scause_out	uint32_t	CSR debug: current scause.
debug_stval_out	uint32_t	CSR debug: current stval.
debug_irq_valid_out	bool	Interrupt debug: selected pending interrupt is takeable.
debug_irq_cause_out	uint32_t	Interrupt debug: selected cause number.
debug_irq_to_supervisor_out	bool	Interrupt debug: selected interrupt delegates to S-mode.
debug_irq_mip_out	uint32_t	Interrupt debug: merged pending bits.
debug_irq_mie_out	uint32_t	Interrupt debug: enable bits from CSR.
debug_irq_mideleg_out	uint32_t	Interrupt debug: delegation bits from CSR.
debug_priv_out	u<2>	CSR debug: current privilege mode.
debug_ra_out	uint32_t	Register debug: current x1/RA value.
debug_regs_write_out	bool	Register debug: writeback wants to write a register.
debug_regs_write_actual_out	bool	Register debug: writeback is not blocked by memory wait.

Port	Type	Description
debug_regs_wr_id_out	uint8_t	Register debug: destination register index.
debug_regs_data_out	uint32_t	Register debug: writeback data.
debug_branch_taken_now_out	bool	Execute debug: branch taken this cycle.
debug_branch_target_now_out	uint32_t	Execute debug: resolved branch target.
debug_decode_instr_out	uint32_t	Decode debug: instruction word entering decode.
debug_decode_pc_out	uint32_t	Decode debug: PC entering decode.
debug_decode_br_out	uint8_t	Decode debug: branch operation field.
debug_decode_imm_out	uint32_t	Decode debug: decoded immediate.
sbi_set_timer_out	bool	Local emulation of legacy SBI set_timer ECALL.
sbi_timer_lo_out	uint32_t	Low 32 bits of the requested SBI timer compare value.
sbi_timer_hi_out	uint32_t	High 32 bits of the requested SBI timer compare value.
reset_pc_in	uint32_t	Reset PC.
boot_hartid_in	uint32_t	Reset value for a0, conventionally hart id.
boot_dtb_addr_in	uint32_t	Reset value for a1, conventionally device-tree address.
boot_priv_in	u<2>	Initial privilege mode when CSR support is enabled.
memory_base_in	uint32_t	Physical base address of the attached memory map.
memory_size_in	uint32_t	Total byte size of RAM plus IO region visible to L2.
mem_region_size_in	uint32_t[L2_MEM_PORTS]	Cumulative sizes of the four L2 memory/device regions.

Port	Type	Description
`clint_msip_in`	`bool`	CLINT machine software interrupt pending input, when interrupts are enabled.
`clint_mtip_in`	`bool`	CLINT machine timer interrupt pending input, when interrupts are enabled.
`external_irq_in`	`bool`	External interrupt input, normally driven by PLIC when interrupts are enabled.
`axi_in`	`Axi4If<clog2(MAX_RAM_SIZE), 4, TRIBE_L2_AXI_WIDTH>[L2_MEM_PORTS]`	External coherent AXI master access into L2. Mostly DMA-style devices.
`axi_out`	`Axi4If<clog2(MAX_RAM_SIZE), 4, TRIBE_L2_AXI_WIDTH>[L2_MEM_PORTS]`	L2 master ports toward RAM and device regions.
`perf_out`	`TribePerf`	Per-cycle performance/stall/cache debug snapshot.
`debugen_in`	`bool`	C++ simulation debug print enable flag.

The main combinational control in `Tribe` handles pipeline hazards, branch redirects, global memory wait, MMU page-table-walk arbitration, trap redirection, SBI timer emulation, I-cache/TLB invalidation, and late load forwarding. The DMMU and IMMU page-table walkers share the L2 data-side port with normal data-cache traffic; DMMU requests have priority over IMMU requests after normal data-cache reads/writes. The data MMU has a direct physical window for the IO region so MMIO is not translated by Sv32.

`Tribe` itself does not contain the UART, CLINT, PLIC, DRAM, or accelerator. It exposes the L2 AXI master/slave ports and interrupt inputs needed by wrappers. The default simulation wrapper and the SoC wrapper connect those ports to concrete devices.

Decode

Ports:

Port	Type	Description
pc_in	uint32_t	PC associated with instr_in.
instr_valid_in	bool	Indicates that instr_in is valid for decode.
instr_in	uint32_t	32-bit instruction word from I-cache; compressed instructions are decoded from low bits.
regs_data0_in	uint32_t	Register-file value for decoded rs1.
regs_data1_in	uint32_t	Register-file value for decoded rs2.
rs1_out	u<5>	Decoded source register 1 index for register-file read.
rs2_out	u<5>	Decoded source register 2 index for register-file read.
state_out	State	Decoded State carrying operands and control fields into execute.

Decode selects the active decoder specification from Rv32im, Rv32ia, and Zicsr according to build flags. It fills the pipeline State, attaches the current PC, marks validity from instr_valid_in, fetches register values, and handles the AUIPC PC operand special case. It is combinational and has no internal register state.

Execute

Ports:

Port	Type	Description
state_in	State	Execute-stage State after forwarding and trap redirection.
alu_result_out	uint32_t	ALU result. For comparisons, upper result bits carry equality information used by branches.
debug_alu_a_out	uint32_t	Debug ALU operand A.
debug_alu_b_out	uint32_t	Debug ALU operand B.
branch_taken_out	bool	Branch/jump taken result.

Port	Type	Description
branch_target_out	uint32_t	Resolved branch or jump target.

Execute contains the ALU and branch decision logic. It supports integer arithmetic, shifts, comparisons, multiply/divide operations, address calculation for memory operations, and branch target calculation. Trap and return redirection are represented as branch-like state before this stage enters Execute.

ExecuteMem

Ports:

Port	Type	Description
state_in	State	Execute-stage State after forwarding/trap-return adjustments.
alu_result_in	uint32_t	Effective address or ALU result from Execute.
dcache_read_valid_in	bool	Atomic read response valid, when atomics are enabled.
dcache_read_addr_in	uint32_t	Address tag returned with D-cache read data.
dcache_read_expected_addr_in	uint32_t	Expected physical address for atomic read completion.
dcache_read_data_in	uint32_t	D-cache read data used by AMO operations.
mem_stall_in	bool	Holds issued memory request while D-cache/L2 cannot accept it.
hold_in	bool	Holds request metadata while the pipeline waits for writeback.
mem_write_out	bool	Registered store request to D-cache.
mem_write_addr_out	uint32_t	Store address to D-cache.
mem_write_data_out	uint32_t	Store data to D-cache.
mem_write_mask_out	uint8_t	Store byte mask to D-cache.
mem_read_out	bool	Registered load request to D-cache.
mem_read_addr_out	uint32_t	Load address to D-cache.

Port	Type	Description
mem_split_out	bool	Current access crosses a 32-byte L1 line and must be split.
mem_split_busy_out	bool	A delayed second split transaction is pending.
split_load_out	bool	Writeback must assemble a split load from two words.
split_load_low_out	uint32_t	Low aligned address for split-load matching.
split_load_high_out	uint32_t	High aligned address for split-load matching.
atomic_busy_out	bool	Atomic LR/SC/AMO sequence is active, when atomics are enabled.
atomic_sc_result_out	uint32_t	Store-conditional architectural result, 0 for success and 1 for failure.

ExecuteMem turns decoded memory operations into D-cache requests. It detects accesses that cross an L1 cache line and issues the first and second aligned transactions in order, while providing metadata for WritebackMem to reconstruct split loads. With ENABLE_RV32IA, it also implements LR/SC reservation tracking and AMO read-modify-write sequencing.

Writeback

Ports:

Port	Type	Description
state_in	State	Writeback-stage State.
alu_result_in	uint32_t	ALU result to write for ALU instructions.
mem_data_in	uint32_t	Raw aligned load data.
mem_data_hi_in	uint32_t	High word for legacy split-load interface; normally zero after WritebackMem assembly.
mem_addr_in	uint32_t	Load address used for byte/halfword interpretation.
mem_split_in	bool	Indicates split-load result selection.

Port	Type	Description
`regs_data_out`	`uint32_t`	Final architectural value for the integer register file.
`regs_wr_id_out`	`uint8_t`	Destination register index.
`regs_write_out`	`bool`	Register-file write enable.

`Writeback` formats the final value for the integer register file. It selects PC+2, PC+4, ALU result, or memory result according to `State::wb_op`, and sign- or zero-extends byte and halfword loads based on `funct3`. Register x0 is still protected by the register file and write-enable logic around this stage.

WritebackMem

Ports:

Port	Type	Description
`state_in`	`State`	Writeback-stage `State`.
`alu_result_in`	`uint32_t`	Address expected for the active load response.
`split_load_in`	`bool`	Indicates that two aligned read responses must be assembled.
`split_load_low_addr_in`	`uint32_t`	Low aligned address of a split load.
`split_load_high_addr_in`	`uint32_t`	High aligned address of a split load.
`dcache_read_valid_in`	`bool`	D-cache read response valid.
`dcache_read_addr_in`	`uint32_t`	D-cache response address tag.
`dcache_read_data_in`	`uint32_t`	D-cache read data.
`dcache_write_valid_in`	`bool`	Store request accepted/visible to D-cache.
`dcache_write_addr_in`	`uint32_t`	Store address for forwarding history.
`dcache_write_data_in`	`uint32_t`	Store data for forwarding history.
`dcache_write_mask_in`	`uint8_t`	Store byte mask for forwarding history.
`hold_in`	`bool`	Keeps pending load information while pipeline is stalled.

Port	Type	Description
load_ready_out	bool	Load result is available for register writeback and forwarding.
load_raw_out	uint32_t	Raw load data before architectural sign/zero extension.
load_result_out	uint32_t	Architecturally extended load value for late forwarding.
wb_mem_data_out	uint32_t	Load data passed to Writeback.
wb_mem_data_hi_out	uint32_t	High split-load word passed to Writeback.

WritebackMem is the one-cycle memory/writeback boundary. It captures D-cache responses, waits until both halves of a split load are available, assembles unaligned words, and performs short store-to-load forwarding. Forwarding is disabled for atomics so AMO semantics are not hidden by local store history.

Peripheral

L1Cache

Ports:

Port	Type	Description
write_in	bool	CPU write request. The data L1 is write-through to L2
write_data_in	uint32_t	32-bit write data.
write_mask_in	uint8_t	Byte mask for the 32-bit write.
read_in	bool	CPU read request.
addr_in	uint32_t	CPU byte address.
read_data_out	uint32_t	32-bit CPU read result.
read_addr_out	uint32_t	Address tag associated with read_data_out.
read_valid_out	bool	Read response valid.
busy_out	bool	Cache cannot accept or complete the current request.
stall_in	bool	Front-end/pipeline stall input.
flush_in	bool	Redirect flush input, used by I-cache fetch.
invalidate_in	bool	Clears valid tags for FENCE.I/SFENCE.VMA.
cache_disable_in	bool	Forces direct, uncached reads for MMIO/direct paths.
mem_write_out	bool	Write-through store request to L2.
mem_write_data_out	uint32_t	L2 write data.

Port	Type	Description
mem_write_mask_out	uint8_t	L2 write byte mask.
mem_read_out	bool	L2 read request for refill or direct read.
mem_addr_out	uint32_t	L2 address for refill or direct read.
mem_read_data_in	logic<PORT_BITWIDTH>	L2 read beat, width PORT_BITWIDTH.
mem_wait_in	bool	L2 wait/hold response.
perf_out	L1CachePerf	L1 hit/wait/state performance snapshot.

L1Cache is a small set-associative cache with 32-byte lines. It is instantiated twice: icache with ID 0 and dcache with ID 1. The line is stored in even/odd 16-bit RAM halves, then assembled into 32-bit words on reads and refills. The data L1 is write-through; it sends 32-bit stores directly to L2 and does not own dirty state.

The L1 refill port width equals the selected L2 AXI width and may be smaller than the cache line. A miss refills a line over one or more PORT_BITWIDTH beats. For data reads that would cross the final word of a cache line, the CPU split logic handles the access before L2 sees a cacheable fill. MMIO/direct reads bypass L1 caching. Cached RAM direct reads stay aligned to the containing L2 beat so dirty data already present in L2 is used instead of stale backing RAM.

L2Cache

Ports:

Port	Type	Description
i_read_in	bool	Instruction-side read request from I-cache.
i_write_in	bool	Instruction-side write request; normally false.
i_addr_in	uint32_t	Instruction-side address.
i_write_data_in	uint32_t	Instruction-side write data.
i_write_mask_in	uint8_t	Instruction-side byte mask.
i_read_data_out	logic<PORT_BITWIDTH>	Instruction-side read beat.
i_wait_out	bool	Instruction-side wait.
d_read_in	bool	Data-side read request from D-cache or MMU page-table walker.
d_write_in	bool	Data-side write request from D-cache.
d_addr_in	uint32_t	Data-side address.
d_write_data_in	uint32_t	Data-side 32-bit write data.
d_write_mask_in	uint8_t	Data-side byte mask.
d_read_data_out	logic<PORT_BITWIDTH>	Data-side read beat.

Port	Type	Description
d_wait_out	bool	Data-side wait.
memory_base_in	uint32_t	Base physical address of the memory map.
memory_size_in	uint32_t	Total visible bytes across all regions.
mem_region_size_in	uint32_t[MEM_PORTS]	Per-region byte sizes. Regions are contiguous, not interleaved.
mem_region_uncached_in	bool[MEM_PORTS]	Per-region bypass flag; device/MMIO regions are uncached.
axi_in	Axi4If<MEM_ADDR_BITS, 4, PORT_BITWIDTH>[MEM_PORTS]	Coherent slave ports for external AXI masters such as DMA.
axi_out	Axi4If<MEM_ADDR_BITS, 4, PORT_BITWIDTH>[MEM_PORTS]	Master ports to RAM/device regions.

L2Cache is a 4-way set-associative shared cache with 32-byte lines and a configurable memory beat width. It arbitrates coherent external AXI slave requests, data-cache requests, and instruction-cache requests onto one L2 tag/data RAM. External AXI masters can read cached data written by the CPU, and the CPU can read data written through an external AXI slave port.

L2 memory ports are split into contiguous regions. Address selection subtracts memory_base_in, chooses a region by cumulative region size, and then forwards a local byte address to the selected AXI master port. Cached regions allocate and evict dirty cache lines through AXI. Uncached regions bypass the tag/data RAM and issue single-beat MMIO reads and writes, so device regions do not infer cache storage.

BranchPredictor

Ports:

Port	Type	Description
lookup_valid_in	bool	Valid decoded branch lookup.
lookup_pc_in	uint32_t	Branch PC for lookup.
lookup_target_in	uint32_t	Newly decoded branch target.
lookup_fallthrough_in	uint32_t	Sequential next PC.
lookup_br_op_in	u<4>	Branch operation type.
predict_taken_out	bool	Predicted branch-taken flag.
predict_next_out	uint32_t	Predicted next PC.

Port	Type	Description
update_valid_in	bool	Execute-stage branch update valid.
update_pc_in	uint32_t	Resolved branch PC.
update_taken_in	bool	Resolved branch direction.
update_target_in	uint32_t	Resolved branch target.

BranchPredictor is a direct-mapped predictor with saturating counters, valid bits, tags, and target storage. Conditional branches use the counter state. JAL, JALR, and JR are predicted taken. On execute resolution, the predictor updates direction and target state.

Interrupt Controller

Ports:

Port	Type	Description
mstatus_in	uint32_t	CSR mstatus bits used for global interrupt enables.
mie_in	uint32_t	CSR interrupt enable mask.
mideleg_in	uint32_t	Machine interrupt delegation mask.
mip_sw_in	uint32_t	Software-writable pending bits from CSR state.
priv_in	u<2>	Current privilege mode.
clint_msip_in	bool	Machine software interrupt from CLINT.
clint_mtip_in	bool	Machine timer interrupt from CLINT.
external_irq_in	bool	External interrupt request, normally PLIC output.
mip_out	uint32_t	Merged interrupt-pending bits.
interrupt_valid_out	bool	An enabled interrupt should be taken.
interrupt_cause_out	uint32_t	Selected interrupt cause number.
interrupt_to_supervisor_out	bool	Interrupt should trap to supervisor mode by delegation.

InterruptController merges hardware CLINT interrupt inputs, external interrupt input, and writable CSR pending bits. It masks pending bits with mie, applies privilege-aware

global enable rules from `mstatus`, applies `mideleg`, and reports one pending cause to the CSR/trap block. The external interrupt path is used by PLIC for Linux UART interrupts.

MMU/TLB

Ports:

Port	Type	Description
`vaddr_in`	`uint32_t`	Virtual address to translate.
`read_in`	`bool`	Load translation request.
`write_in`	`bool`	Store translation request.
`execute_in`	`bool`	Instruction-fetch translation request.
`satp_in`	`uint32_t`	Current `satp`; Sv32 is active when MODE is 1.
`priv_in`	`u<2>`	Current privilege mode.
`direct_base_in`	`uint32_t`	Base of direct physical bypass window.
`direct_size_in`	`uint32_t`	Size of direct physical bypass window.
`fill_in`	`bool`	External/manual TLB fill request.
`fill_index_in`	`u<clog2(ENTRIES)>`	External/manual fill index.
`fill_vpn_in`	`uint32_t`	External/manual fill virtual page number.
`fill_ppn_in`	`uint32_t`	External/manual fill physical page number.
`fill_flags_in`	`uint8_t`	External/manual fill PTE flags.
`sfence_in`	`bool`	Invalidates cached translations.
`mem_read_out`	`bool`	Page-table-walker memory read request.
`mem_addr_out`	`uint32_t`	Physical PTE address requested by the walker.
`mem_read_data_in`	`uint32_t`	32-bit PTE data returned by memory.
`mem_wait_in`	`bool`	Page-table-walker memory wait.
`paddr_out`	`uint32_t`	Translated or bypassed physical address.
`translated_out`	`bool`	Translation is active for the current request.
`hit_out`	`bool`	TLB hit or translation disabled.
`fault_out`	`bool`	Page fault or permission fault.
`miss_out`	`bool`	Translation miss requiring a page-table walk.
`busy_out`	`bool`	Walker is active or waiting.
`debug_last_pte_out`	`uint32_t`	Last PTE captured by the walker.
`debug_last_addr_out`	`uint32_t`	Last PTE address captured by the walker.

`MMU_TLB` implements a small Sv32 TLB with a hardware page-table walker. There are separate instances for instruction fetch and data access. Translation is enabled only when `satp.MODE == 1` and current privilege is not M-mode. A direct mapping window can bypass translation; Tribe uses it for DMMU access to MMIO so device addresses remain physical under an OS.

The walker reads the level-1 PTE from the `satp` root page and, when needed, reads the level-0 PTE. It supports level-1 superpages and level-0 pages, checks valid/write/read combinations, checks A/D/R/W/X/U permissions, and reports instruction/load/store page faults through the main CSR/trap path. `SFENCE.VMA` clears cached translations.

Axi4RegionMux

Ports:

Port	Type	Description
`slave_in`	`Axi4If<ADDR_WIDTH, ID_WIDTH, DATA_WIDTH>`	AXI slave side connected to an L2 uncached/device region.
`masters_out`	`Axi4If<ADDR_WIDTH, ID_WIDTH, DATA_WIDTH>[N]`	AXI master sides connected to devices.
`region_base_in`	`uint32_t[N]`	Device-local base address for each region.
`region_size_in`	`uint32_t[N]`	Byte size of each device region.

`Axi4RegionMux` routes one AXI device-region port to several memory-mapped device responders. Regions are decoded by local address range. The mux preserves AXI channel IDs and returns read/write responses from the selected device. Tribe uses this mux for the IO region behind the L2 uncached port.

Axi4Ram

Ports:

Port	Type	Description
`axi_in`	`Axi4If<ADDR_WIDTH, ID_WIDTH, DATA_WIDTH>`	AXI access to RAM storage.

`Axi4Ram` is the common testbench and SoC RAM responder. It accepts single-beat AXI reads and writes, stores data at the configured beat width, and supports checkpoint serialization through its `_strobe(FILE*)` path in native C++ tests. The default Tribe executable wrapper uses three RAM instances; the SoC wrapper keeps two DRAMs in `SystemTest` and places the third memory region inside `System`.

SoC

`tribe/SoC/System.cpp` defines a `System` DUT and a `SystemTest` wrapper. `System` integrates the `Tribe` core with the on-chip IO space, NS16550A UART, CLINT, PLIC, Accelerator, and the third internal AXI RAM region. `SystemTest` provides external `dram0` and `dram1` RAMs and converts realistic hardware configuration values into `System` input ports before running the same program modes as the corresponding Tribe targets.

`System` ports:

Port	Type	Description
reset_pc_in	uint32_t	Reset PC forwarded to Tribe.
boot_hartid_in	uint32_t	Boot hart id forwarded as initial a0.
boot_dtb_addr_in	uint32_t	Boot DTB address forwarded as initial a1.
boot_priv_in	u<2>	Initial privilege mode.
memory_base_in	uint32_t	Physical base of the memory map.
memory_size_in	uint32_t	Total visible memory-map size.
mem_region_size_in	uint32_t[L2_MEM_PORTS]	Region sizes forwarded to L2.
uart_rx_valid_in	bool	External UART RX byte valid.
uart_rx_data_in	uint8_t	External UART RX byte.
uart_rx_ready_out	bool	UART can accept another RX byte.
uart_tx_valid_out	bool	UART transmitted a byte this cycle.
uart_tx_data_out	uint8_t	UART transmitted byte.
perf_out	TribePerf	CPU performance snapshot.
dmem_write_out	bool	CPU debug data-memory write indication.
dmem_write_data_out	uint32_t	CPU debug data-memory write data.
dmem_write_mask_out	uint8_t	CPU debug data-memory write mask.
dmem_read_out	bool	CPU debug data-memory read indication.
dmem_addr_out	uint32_t	CPU debug data-memory address.
imem_read_addr_out	uint32_t	CPU debug instruction-memory address.
debug_immu_ptw_read_out	bool	Optional IMMU page-table-walk read mirror.
debug_immu_ptw_addr_out	uint32_t	Optional IMMU page-table-walk address mirror.

Port	Type	Description
debug_immu_busy_out	bool	Optional IMMU busy mirror.
debug_immu_fault_out	bool	Optional IMMU fault mirror.
debug_immu_paddr_out	uint32_t	Optional IMMU translated physical address mirror.
debug_immu_last_addr_out	uint32_t	Optional IMMU last PTE address mirror.
debug_immu_last_pte_out	uint32_t	Optional IMMU last PTE value mirror.
debug_icache_read_valid_out	bool	Optional I-cache read-valid mirror.
debug_icache_read_addr_out	uint32_t	Optional I-cache read address mirror.
debug_fetch_valid_out	bool	Optional fetch-valid mirror.
debug_memory_wait_out	bool	Optional memory-wait mirror.
debug_wb_load_ready_out	bool	Optional writeback load-ready mirror.
debug_wb_mem_wait_out	bool	Optional writeback memory-wait mirror.
debug_icache_read_in_out	bool	Optional I-cache read request mirror.
debug_icache_stall_in_out	bool	Optional I-cache stall mirror.
debug_dmmu_ptw_read_out	bool	Optional DMMU page-table-walk read mirror.
debug_dmmu_ptw_addr_out	uint32_t	Optional DMMU page-table-walk address mirror.
debug_dmmu_busy_out	bool	Optional DMMU busy mirror.
debug_dmmu_fault_out	bool	Optional DMMU fault mirror.
debug_mmu_ptw_word_out	uint32_t	Optional page-table-walker word mirror.
debug_pc_out	uint32_t	Optional current PC mirror.
debug_satp_out	uint32_t	Optional `satp` mirror.
debug_mstatus_out	uint32_t	Optional `mstatus` mirror.
debug_mtvec_out	uint32_t	Optional `mtvec` mirror.
debug_mepc_out	uint32_t	Optional `mepc` mirror.

Port	Type	Description
debug_mcause_out	uint32_t	Optional mcause mirror.
debug_mtval_out	uint32_t	Optional mtval mirror.
debug_sepc_out	uint32_t	Optional sepc mirror.
debug_stvec_out	uint32_t	Optional stvec mirror.
debug_scause_out	uint32_t	Optional scause mirror.
debug_stval_out	uint32_t	Optional stval mirror.
debug_priv_out	u<2>	Optional privilege-mode mirror.
debug_ra_out	uint32_t	Optional x1/RA mirror.
debug_regs_write_out	bool	Optional register write-request mirror.
debug_regs_write_actual_out	bool	Optional unblocked register write mirror.
debug_regs_wr_id_out	uint8_t	Optional register destination mirror.
debug_regs_data_out	uint32_t	Optional register write-data mirror.
debug_branch_taken_now_out	bool	Optional branch-taken mirror.
debug_branch_target_now_out	uint32_t	Optional branch-target mirror.
debug_decode_instr_out	uint32_t	Optional decode instruction mirror.
debug_decode_pc_out	uint32_t	Optional decode PC mirror.
debug_decode_br_out	uint8_t	Optional decode branch-operation mirror.
debug_decode_imm_out	uint32_t	Optional decode immediate mirror.
axi_out	Axi4If<clog2(MAX_RAM_SIZE), 4, TRIBE_L2_AXI_WIDTH>[2]	External AXI master ports for dram0 and dram1 in SystemTest.

The SoC memory map mirrors the default Tribe executable wrapper: two external DRAM regions, one internal RAM region, and one uncached IO region. Inside IO, Axi4RegionMux maps UART at offset 0x0, CLINT at offset 0x100, Accelerator at offset 0xC100, and PLIC at offset 0x10000. PLIC source 1 is driven by the NS16550A UART IRQ and then routed back to Tribe.external_irq_in.

Devices

Device modules live in `tribe/devices`. They are memory-mapped AXI responders, except `Accelerator`, which also owns an AXI master DMA port. In the default Tribe simulation wrapper and in `System`, devices are placed behind an IO-space region mux connected to the uncached L2 IO region.

IOUART

Ports:

Port	Type	Description
`axi_in`	`Axi4If<ADDR_WIDTH, ID_WIDTH, DATA_WIDTH>`	MMIO register access.
`uart_valid_out`	`bool`	One-cycle pulse when a byte is writt
`uart_data_out`	`uint8_t`	Byte written by software.

`IOUART` is a minimal UART-like output device. It exposes `TXDATA` at offset `0x00` and `STATUS` at offset `0x04`; status bit 0 is always ready. Writes to `TXDATA` emit `uart_valid_out` and `uart_data_out`. It is useful for simple bare-metal tests.

NS16550A

Ports:

Port	Type	Description
`axi_in`	`Axi4If<ADDR_WIDTH, ID_WIDTH, DATA_WIDTH>`	MMIO register access to the 16550-style register file.
`uart_valid_out`	`bool`	One-cycle pulse when software writes the transmit holding register.
`uart_data_out`	`uint8_t`	Transmitted byte.
`uart_rx_valid_in`	`bool`	External RX byte valid.
`uart_rx_data_in`	`uint8_t`	External RX byte.
`uart_rx_ready_out`	`bool`	Single-byte RX buffer is empty and can accept a byte.
`irq_out`	`bool`	UART interrupt request, used as PLIC source 1 in the wrappers.

`NS16550A` models enough of a 16550-compatible UART for firmware and Linux console probing. It implements the usual register offsets for RBR/THR/DLL, IER/DLM, IIR/FCR,

LCR, MCR, LSR, MSR, and SCR. Transmit is modeled as always empty and ready by setting LSR.THRE and LSR.TEMT; THR-empty interrupts are intentionally not generated, so Linux can use polling for TX without interrupt storms. Receive uses a one-byte buffer, sets LSR.DR, returns IIR=0x04 when RX interrupt enable is set, clears RBR on read, and asserts irq_out when MCR.OUT2 and IER.RDI are enabled. DLAB selects divisor latch registers at offsets 0 and 1.

CLINT

Ports:

Port	Type	Description
axi_in	Axi4If<ADDR_WIDTH, ID_WIDTH, DATA_WIDTH>	MMIO access to CLINT registers.
set_mtimecmp_in	bool	Direct timer-compare update from local SBI set_timer emulation.
set_mtimecmp_lo_in	uint32_t	Low 32 bits for direct mtimecmp update.
set_mtimecmp_hi_in	uint32_t	High 32 bits for direct mtimecmp update.
msip_out	bool	Machine software interrupt pending.
mtip_out	bool	Machine timer interrupt pending.

CLINT implements the basic RISC-V local interrupt timer registers used by Tribe: msip, mtimecmp, and mtime. The timer increments according to TRIBE_CLINT_TICK_DIV_CONFIG, which lets Linux runs slow the timer relative to CPU model cycles. mtip_out is asserted when mtime >= mtimecmp; msip_out follows bit 0 of the msip register. The direct set_mtimecmp_* inputs let the core emulate legacy SBI timer calls without requiring firmware support.

PLIC

Ports:

Port	Type	Description
axi_in	Axi4If<ADDR_WIDTH, ID_WIDTH, DATA_WIDTH>	MMIO access to PLIC priority, pending, enable, threshold, and claim/complete registers.

Port	Type	Description
source_irq_in	bool[SOURCES]	Level-sensitive interrupt source inputs. Source 0 is unused by convention.
external_irq_out	bool	External interrupt line to the CPU interrupt controller.

PLIC is a compact one-context platform interrupt controller. It implements source priority registers, a pending register, one enable register, one threshold register, and a claim/complete register at the standard PLIC-style offsets. Device source levels are latched by small gateway state so a request remains claimable until the CPU reads the claim register. Writing the claimed source ID to the complete register releases the gateway, allowing a still-asserted device level to pend again on a later cycle. In the current wrappers, source 1 is connected to NS16550A.irq_out, and external_irq_out drives Tribe.external_irq_in.

Accelerator

Ports:

Port	Type	Description
axi_in	Axi4If<ADDR_WIDTH, ID_WIDTH, DATA_WIDTH>	CPU MMIO control/status and accelerator-local memory access.
dma_out	Axi4If<ADDR_WIDTH, ID_WIDTH, DATA_WIDTH>	DMA memory access through an L2 coherent slave port.

Accelerator is a test device with a small local word memory, a PRBS generator, and a DMA engine. Its control registers include source address, destination address, transfer length, control, status, and PRBS seed. The DMA path is a real AXI master port and is intended to connect to an L2 slave/coherency port, not to a private CPU shortcut. It can copy main memory into accelerator memory, copy accelerator memory back to main memory, and fill its local memory from a PRBS sequence for software-visible data movement tests.